Highlights

- Open-source Python framework for forest estate decision support.
- Solver-agnostic Model I generator with parallel LP build.
- Native libCBM linkage for carbon stock and flux accounting.
- Hybrid aspatial-to-raster workflows with reproducible geospatial I/O.
- Deterministic reproduction package and archived scaling benchmarks.

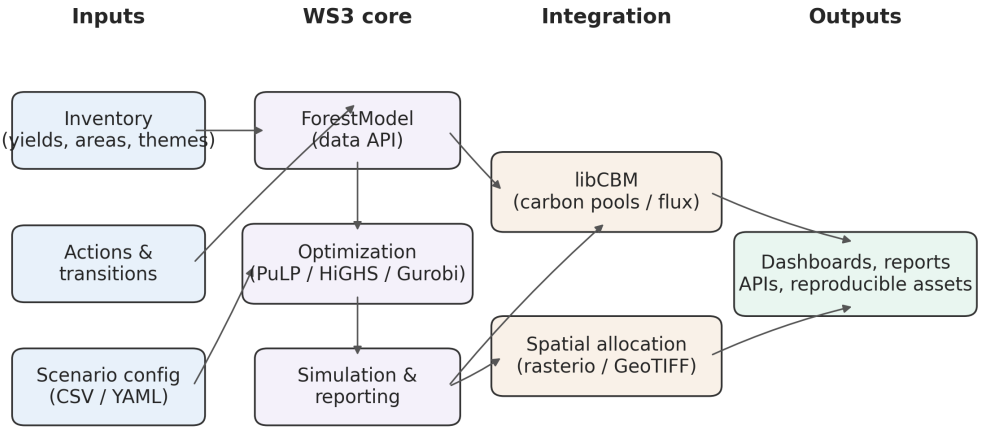
Software availability

- Software name: WS3 (Wood Supply Simulation and Scheduling)
- Developers: Gregory Paradis (UBC FRESH) with contributions from the FRESH lab community
- First public release: 2015 (v0.1); current version: 1.0.4
- Programming language: Python (¿=3.9); optional extras: libcbm, PuLP, gurobipy
- Source: `https://github.com/UBC-FRESH/ws3`; Issue tracker and discussions hosted on GitHub
- Distribution: PyPI package `ws3` (`pip install ws3` or `ws3[cbm]`) with wheels for Linux, macOS, Windows
- Documentation: `https://ws3.readthedocs.io`
- Archive: Zenodo release [29] (DOI 10.5281/zenodo.17219651)
- License: MIT

Graphical Abstract

WS3: An open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems

Gregory Paradis



WS3 coordinates data preparation, optimization, and analysis in a transparent pipeline.

WS3: An open-source Python framework for integrated simulation and optimization of forest landscape and wood supply systems

Gregory Paradis[a,*]

[a]*Faculty of Forestry, University of British Columbia, Canada*

Abstract

Transparent, reproducible decision support for forest landscape planning increasingly requires integrated treatment of harvest scheduling, spatial allocation, and greenhouse gas (GHG) accounting. We present WS3, an open-source Python framework for long-horizon wood supply modeling that combines modular simulation with solver-backed optimization and hybrid spatial/aspatial workflows. WS3 represents stands, actions, and transitions through a clear data model, supports scenario-based analyses, and interoperates with geospatial rasters for spatial allocation of aspatial schedules. A key contribution is a built-in linkage to the open-source Canadian Forest Service Carbon Budget Model (libCBM), enabling seamless carbon stock and flux estimation alongside traditional harvest and area outcomes. We demonstrate WS3 through reproducible use-case demonstrations that couple optimization-driven harvest scheduling with libCBM carbon accounting and a raster allocation step to visualize spatial implications. These demonstrations highlight how alternative policy constraints influence harvest flows and carbon dynamics, illustrating the framework's utility for climate change mitigation planning. The full code, documentation, and notebooks are publicly available via GitHub and PyPI, with a versioned archive on Zenodo [29], and a complete reproduction package accompanying this article. WS3 lowers barriers to rigorous, extensible analysis, supporting researchers and practitioners seeking open, auditable workflows for forest sector planning under evolving climate and policy objectives.

Keywords: Environmental modelling, Decision support, Forest estate, Optimization, Simulation, Spatial modelling, Reproducibility, Open-source software

*Corresponding author: gregory.paradis@ubc.ca

1. Introduction

Forested landscapes now sit at the centre of intertwined climate mitigation, biodiversity, and bioeconomy mandates, and planners are being asked to reconcile wood supply security with measurable reductions in greenhouse-gas emissions across multi-decadal horizons [3, 13, 10]. Meeting these expectations demands decision-support systems that can span harvest scheduling, regeneration and silviculture options, spatial footprint analysis, and explicit carbon accounting, all while remaining transparent enough for regulators, Indigenous governments, and industrial partners to audit [23, 34]. The resulting analytical load has outgrown the ad hoc spreadsheets and siloed software stacks still common in practice, underscoring the need for integrated, reproducible modelling platforms that uphold emerging open-science norms such as the PERFICT principles for predictive ecology [17, 24, 25].

Strategic forest planning has long relied on operations-research formulations such as Model I path scheduling and its variants, which underpin allowable-cut analyses and forest estate studies worldwide [19, 14]. In practice these ideas are delivered through proprietary toolchains—Remsoft Woodstock, Patchworks, FPS-Atlas, JLP (MELA), and Heureka among others—that package data models, solvers, and reporting inside closed desktop environments [32, 35, 11, 27, 18]. While these systems are mature and widely trusted, their licensing, opaque configuration formats, and limited automation support constrain transparency, collaborative extension, and reproducible research workflows increasingly required by regulators and journals [17].

In response, an open-source ecosystem has begun to chip away at these constraints. Spatial simulation platforms such as SpaDES and LANDIS-II, scenario-management frameworks like SyncroSim, and integrative planning prototypes such as PRISM demonstrate strong community appetite for modular, shareable tooling [8, 22, 1, 28]. SpaDES in particular mirrors WS3's open-science commitments—the core team helped write the PERFICT guidance—yet, by design, it delivers a coordination shell that depends on user-supplied simulation modules for every landscape process. That architecture excels at orchestrating high-fidelity spatial and stochastic dynamics, but the developers themselves acknowledge that harvest modules rarely move beyond stylised heuristics. They have therefore started collaborating with us on a `spades_ws3` wrapper so SpaDES users can bring professional-grade wood-supply scheduling into cumulative-effects experiments. LANDIS-II and SELES occupy similar ecological-simulation niches but remain harder to adapt for rigorous forest-estate analysis: LANDIS-II's governance, C# codebase, and Windows-centric binaries slow outside

contributions and impede deployment to linux-based cloud or HPC environments, while SELES is closed-source black-box commercial software whose advanced modules typically require direct support from its original developer. In practice, practitioners must still bolt on their own solver integrations, carbon-accounting pipelines, and spatial reporting to assemble a full decision-support stack [9, 23]. This persistent gap between open landscape-simulation scaffolds and purpose-built forest-estate planning motivates the deterministic, optimization-first framework we introduce with WS3 in the following section.

WS3 fills this methodological gap with a fully open Python framework that preserves the expressive power of legacy Model I scheduling while embedding reproducibility, automation, and carbon accounting as first-class design goals. It imports Woodstock-format text files to protect historical investments, provides solver-agnostic optimization through PuLP/HiGHS with optional Gurobi, couples natively to the Canadian Forest Service's libCBM implementation, and exposes hybrid spatial–aspatial workflows compatible with geospatial rasters and SpaDES-based simulations [32, 26, 16, 15, 21, 4, 8]. The codebase embraces open-science practices—version control, packaged releases, and documented reproduction scripts—to support transparent collaboration across institutions [17, 24].

The remainder of this article details the WS3 architecture and data model (Section 2), demonstrates a reproducible use-case workflows linking solver-backed optimal harvest scheduling to libCBM carbon accounting and spatial disturbance allocation for hypothetical linkage to downstream raster-based spatial simulation models (Section 3), discusses impact and reusability (Section 4), compares WS3 with alternative tools (Section 5), and summarises availability, authorship, and future work to facilitate uptake. Together these sections document (i) the design principles and modular implementation of WS3, (ii) an end-to-end workflow that pairs optimization with carbon analysis and spatial outputs, and (iii) comparative evidence and resources that support reuse by researchers and practitioners.

2. Software description

2.1. Architecture

WS3 implements a modular architecture organized around five roles: (i) core data abstractions; (ii) forest model construction and simulation; (iii) optimization; (iv) spatial allocation; and (v) common utilities. The main modules are:

- forest: defines the ForestModel and core abstractions for development types (dtypes), actions, transitions, yields, and scenario compilation. A scenario is specified by a planning horizon, period length, an initial inventory (areas by dtype and age), and transition rules governing state changes under actions and growth.
- opt: provides a solver-agnostic interface to formulate linear programming harvest-scheduling problems (objective, variables, constraints) and to solve them using PuLP/HiGHS by default, with optional Gurobi.
- spatial: implements utilities to map aspatial schedules to rasters (e.g., GeoTIFF) for visualization and spatial policy evaluation (hybrid spatial/aspatial workflow).
- core/common/financial/forest_helper: shared utilities for interpolation and curves, convenience helpers for examples, and optional financial indicators.

Data model and workflow. The ForestModel class is organized around discrete time (planning periods) and discrete age classes per dtype. Dtypes are keyed by tuples of theme values (e.g., analysis unit, leading species, site class), enabling masks for selective compilation and scheduling. Yields are represented by curves/interpolators, and actions (e.g., harvest, planting, fuel treatment, fire) expose operability and transitions (age reset, development type change, growth updates). WS3 builds a state-transition tree per development type across periods. Users can: (i) schedule heuristically using area-control selectors (e.g., greedy oldest-first), or (ii) schedule optimally via a generic and flexible Model I linear programming model formulation. Indicators are compiled as:

- Action-dependent flows via `compile_product(period, expr, acode=...)` (e.g., harvested volume using a utilization-adjusted expression on the total volume yield).
- Inventory stocks via `inventory(period, yname=...)` (e.g., growing stock).

Interoperability. WS3 can import Woodstock-format text sections (LAN/ARE/YLD/ACT/TRN), enabling reuse of established data pipelines. For carbon, ForestModel exposes `to_cbm_sit` to export Standard Input Table (SIT) configuration/tables consumable by libCBM, with user-provided disturbance-type mappings and per-dtype last-pass-disturbance metadata. Geospatial I/O uses standard formats (shapefile, GeoTIFF). All tabular inputs are plain text (CSV/TSV), and workflows are scripted or notebook-based for reproducibility.

Figure 1 summarizes modules and data flow; Listing 1 outlines the end-to-end case-study workflow used later in the manuscript.

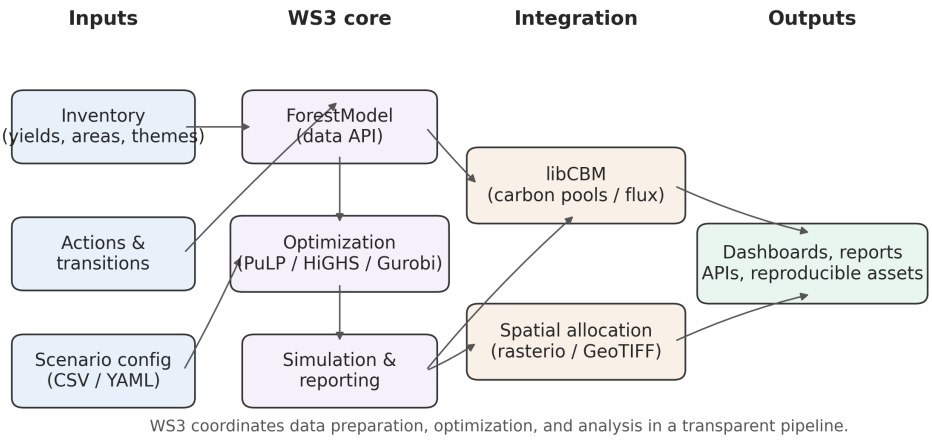


Figure 1: WS3 architecture: inputs feed a ForestModel; scenarios are scheduled via heuristic or LP; results feed libCBM and optional spatial allocation; plots/tables generated for analysis.

Contributions. WS3 delivers four primary contributions:

(i) A solver-agnostic, path-based Model I generator embedded within an audited forest estate API, with parallel coefficient compilation for large instances.

(ii) A first-class libCBM interface that exports Standard Input Tables and ingests carbon pools and fluxes for integrated mitigation accounting.

(iii) A hybrid aspatial-to-raster allocation workflow that emits reproducible geospatial artefacts from deterministic schedules.

(iv) A FAIR-aligned reproduction package, including scaling benchmarks on real inventories and archived releases with pinned environments.

2.2. Optimization formulation (Model I)

WS3 adopts a classic Model I linear programming model formulation [19, 14] in which each decision variable selects the proportion of a development type (all stands with the same initial

Listing 1: Case-study workflow executed by the reproduction package.

```
def run_case_study(data_root, solver="highs"):
    inputs = load_inputs(data_root)
    forest = build_forest_model(inputs)
    schedule = schedule_harvest(forest, method=solver)
    sit_tables = compile_to_cbm(schedule)
    cbm_outputs = run_libcbm(sit_tables)
    spatial_products = allocate_spatial(schedule)
    generate_plots_and_tables(schedule, cbm_outputs, spatial_products)

run_case_study("papers/ems")
```

stratification variable values) allocated to a feasible root-to-leaf prescription (path) across the planning horizon. Let:

I : set of spatial zones (dtypes)

J_i : set of feasible prescriptions (paths) for zone $i \in I$

O : set of outputs (e.g., harvest area, harvest volume, growing stock, habitat)

$O' \subseteq O$: targeted outputs subject to even-flow constraints

T : set of planning periods

$T'_p \subseteq T$: periods on which even-flow for output $p \in O'$ is enforced

Decision variables are proportions:

$$x_{ij} \in [0, 1] \quad \text{for all } i \in I,\ j \in J_i,$$

meaning "the fraction of zone i assigned to prescription j". Let μ_{ijot} denote the quantity of output $o \in O$ in period $t \in T$ produced by allocating x_{ij} to path $j \in J_i$ for zone $i \in I$. Define a reference-period level $y_p := \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt_p^R} x_{ij}$ for each targeted output $p \in O'$ at $t_p^R \in T$, and ε_p as the allowable even-flow tolerance. With general lower/upper bounds v_{ot}^-, v_{ot}^+ on outputs, the Model I

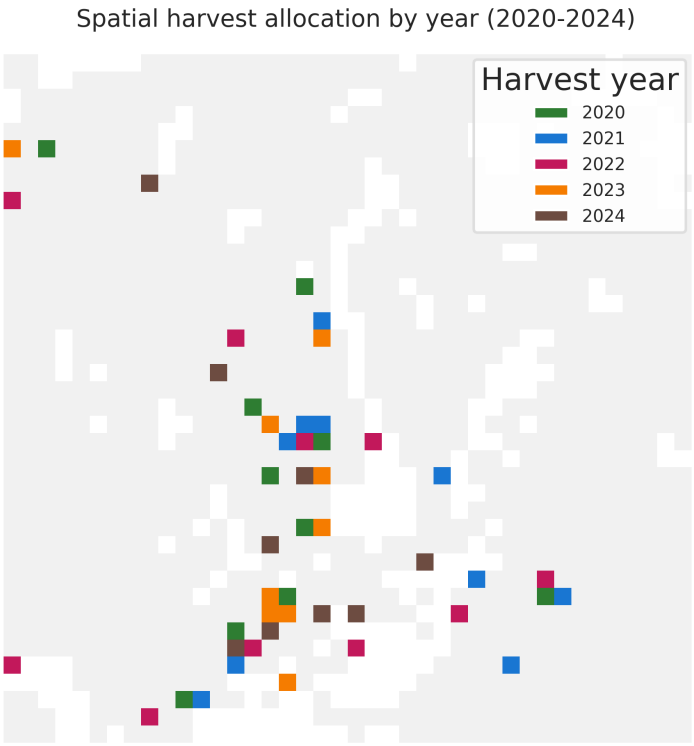


Figure 2: Heuristic schedule allocated to space for the 2020–2024 planning window. Colours denote the first year a cell is harvested, overlaid on the full inventory extent (light grey).

LP is:

$$\text{maximize} \quad \sum_{i \in I} \sum_{j \in J_i} c_{ij}\, x_{ij} \tag{1}$$

$$\text{subject to} \quad (1-\varepsilon_p)\, y_p \le \sum_{i \in I} \sum_{j \in J_i} \mu_{ijpt}\, x_{ij} \le (1+\varepsilon_p)\, y_p, \qquad \forall p \in O',\ \forall t \in T'_p \tag{2}$$

$$v^-_{ot} \le \sum_{i \in I} \sum_{j \in J_i} \mu_{ijot}\, x_{ij} \le v^+_{ot}, \qquad \forall o \in O,\ \forall t \in T \tag{3}$$

$$\sum_{j \in J_i} x_{ij} = 1, \qquad \forall i \in I \tag{4}$$

$$0 \le x_{ij} \le 1, \qquad \forall i \in I,\ \forall j \in J_i. \tag{5}$$

The objective coefficients c_{ij} encode the user-defined value of a path (e.g., total or discounted volume, revenues, multi-criteria scores). Constraints (2) enforce even-flow on targeted outputs, (3) set general lower/upper bounds per period, (4) ensures convex mixing of paths to fully cover each zone, and (5) bounds variables as proportions. In WS3, μ-coefficients arise from replaying each path through the simulator, calling `compile_product` (for action-dependent outputs like harvest area/volume) or `inventory` (for stocks like growing stock). The LP matrix is extremely sparse because each path contributes to a limited set of outputs and periods.

Code-to-math mapping. WS3 exposes a coefficient-function pattern that compiles objective and constraint rows from paths. For example, helper functions like `cmp_c_z` (objective), `cmp_c_caa` (action-based flows), and `cmp_c_ci` (inventory-based constraints) traverse path nodes (period, action code, dtype key, age), evaluate the corresponding μ-like contributions via `compile_product`/`inventory`, and assemble the LP using `fm.add_problem(...)`. Users select the solver (HiGHS by default; Gurobi optional) and solve via the standard PuLP interface. Upon optimality, WS3 compiles and applies the schedule, then re-simulates to produce indicators for analysis.

Design implications. This path-based Model I approach embeds intertemporal feasibility in each column, separates simulation from optimization, and enables straightforward addition of new outputs/constraints by supplying new coefficient functions. It also aligns with established forest-planning literature while remaining solver-agnostic and highly sparse for scalability.

2.3. Implementation

Technology stack. WS3 is implemented in Python and distributed via PyPI. Optimization problems are formulated with PuLP [26] and solved with open-source HiGHS [16] by default, with optional Gurobi [15] for larger instances. Geospatial I/O uses rasterio/fiona/GeoPandas. Documentation (Sphinx) and continuous integration ensure API consistency, example executability, and unit testing.

libCBM integration. The ForestModel exposes a to_cbm_sit method that compiles a SIT configuration and tables (classifiers, inventory, yields, disturbance events, transitions) consumable by libCBM. Users provide a disturbance type mapping and last-pass disturbance metadata to ensure correct dead organic matter (DOM) spin-up. The sequential demonstration runs libCBM for 200 years using the official Python API and documentation [6, 5].

Reproducibility and packaging. The repository includes examples and a reproduction package

(papers/ems/repro) with pinned requirements and scripts to generate all figures and tables. Versioned releases are archived on Zenodo [29]. WS3 supports interactive (Jupyter) and batch (Linux) workflows.

2.4. Quality assurance and benchmarking

Automated testing. WS3 ships with a pytest suite that exercises the common, core, financial, forest, and optimization modules (29 tests). The suite runs in approximately 4 s on a Linux workstation (Python 3.12) and is executed for every push and pull request via a GitHub Actions workflow (Ubuntu, Python 3.10) that also regenerates a coverage badge [38]. Style tooling (black, ruff) is bundled in `requirements-dev.txt` and wired into the development makefile, ensuring consistent formatting before release.

Input stewardship and validity. WS3, like its Woodstock heritage, makes no assumptions about inventories, yields, operability, objectives, or constraints. The space of "valid input" is project- and policy-specific and inherently high dimensional; exhaustive auto-validation is out of scope. WS3 provides targeted checks in high-leverage paths (e.g., fallback to template transitions when age-specific rules are missing; basic range/shape sanity checks) and clear documentation of required structures for inventories, yields, actions, and transitions. Responsibility for input quality and modeling assumptions rests with qualified analysts; WS3 is a professional framework rather than a prescriptive application.

Determinism and reproducibility. Given identical inputs, WS3 produces identical outputs. Minor nondeterminism is limited to optional solver seeds and randomized block ordering in the spatial allocation utility; both are controllable and fixed in the reproduction scripts. Consequently, statistical variance on deterministic time series is not meaningful; we report solved status, model scale, and runtime/throughput metrics instead. The reproduction package pins versions and seeds to yield byte-identical artifacts across runs.

Verification against Woodstock. WS3 was originally verified against Woodstock (2015) by replaying identical datasets and comparing action-specific flows and stocks across periods, yielding functionally equivalent outputs for the implemented subset of features. Known departures from Woodstock semantics are flagged in code and documentation. Ongoing development follows a trust-but-verify pattern: changes to core loops are regression-tested against known-good benchmarks before release.

Case-study repro checks. The EMS reproduction package encapsulates the sequential WS3→libCBM pipeline in `make_repro.sh`. Inside the project's reference LXD container (Ubuntu 24.04 on dual Xeon 6254 CPUs, 72 logical cores, 768 GB PC-23400 RAM) the script completes in 6.1 s, re-creating Figures 6–7, Figure 2, and the tables stored in `papers/ems/tables`. The workflow is deterministic: running `generate_case_study.py` a second time produces byte-identical CSV and PNG assets, providing an integration test for the scheduling, SIT export, and libCBM coupling.

Performance spot checks. The heuristic scheduler used in the case study produces a 100-year plan in under a second, and the subsequent 200-year libCBM simulation dominates total runtime (˜5 s). Larger linear programs formulated through `ws3.opt` benefit from HiGHS (default) or optional Gurobi: internal regression notebooks (e.g., Examples 030/040) routinely solve tens-of-thousands-of-stand path-based instances within minutes using HiGHS, while the optional Gurobi backend shortens solve time further via its advanced presolve and feasibility tools. Raster allocations scale linearly with the number of cells processed per period and are trivially parallelizable.

Optional scalability experiment. To characterize scaling on real inventories, the reproduction package optionally integrates a DataLad-hosted benchmark dataset comprising five non-overlapping timber supply areas (TSAs) in northeastern British Columbia (the same study region as Boisvenue et al. 2). Enabling the optional step (`RUN_SCALING=1`) fetches the dataset and executes `papers/ems/repro/run_scaling_benchmarks.py`, which sorts TSAs by complexity and evaluates them individually and cumulatively (`tsa08`, `tsa08+tsa40`, . . . , `tsa08+tsa40+tsa41+tsa24+tsa16`). The scripted benchmark always runs the heuristic scheduler on a single worker—reflecting the implementation's serial design—and records sequential spatial-allocation runtimes using `ForestRaster`. On the reference container the heuristic schedules complete in 1.3–31.4 s while spatial allocation requires 46–656 s as problem size grows from 4.7 k to 37.7 k dtype–age pairs. Setting `RUN_LP=1` extends the run to model-building benchmarks for the deterministic Model I LP: WS3 constructs the LP with 1 and 16 workers (parallel coefficient compilation) and solves it with matching HiGHS thread counts, logging build/solve time, solver status, and stage-wise memory footprints. LP builds scale from 5.1 s (4.7 k dtype–age pairs) to 92.8 s (37.7 k dtype–age pairs) with single-worker compilation, while the 16-worker mode trims build time to 59 s at the cost of higher peak RSS (2.5 GB versus 1.8 GB). Because large path-based LPs routinely exceed tens of gigabytes of RAM on even larger inventories, the LP measurements are opt-in and intended for suitably provisioned systems. All metrics are written to `perf_scaling.csv` with deterministic seeds so that regenerated figures

and tables remain byte-identical.

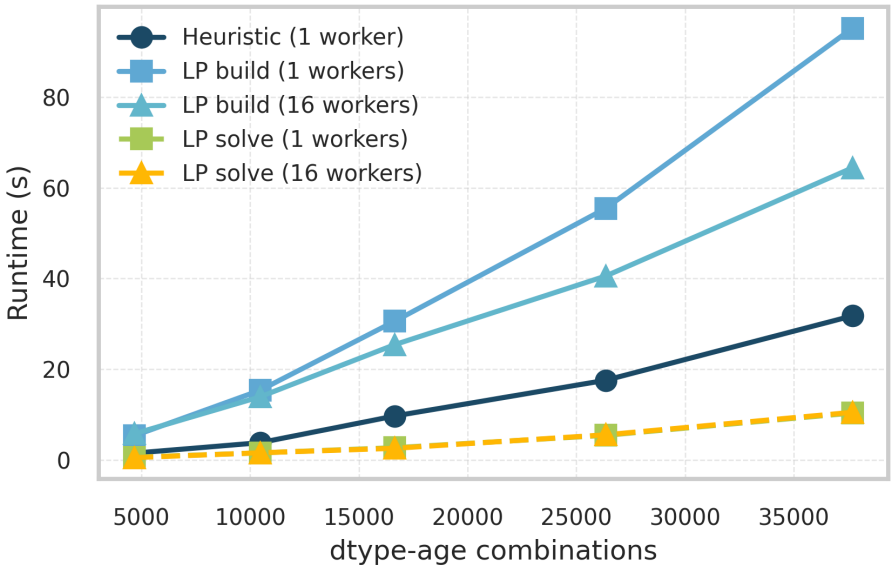


Figure 3: Scheduling runtime versus problem size. Heuristic runs are single worker; LP build/solve lines appear when `RUN_LP=1`.

3. Illustrative use-case demonstrations

We open with a two-stage sequential pipeline demonstration that first schedules harvest in WS3 and then evaluates carbon dynamics using libCBM. The workflow replicates the existing example 031_ws3_libcbm_sequential-builtin.ipynb to ensure full reproducibility.

Study design. We use the public example dataset `tsa24_clipped` provided with WS3. The inputs are stored as Woodstock-format text files (landscape, areas, yields, actions, transitions) under `examples/data/woodstock_model_files` and imported into a `ForestModel`. We then schedule harvest using a self-parameterizing area-control heuristic to produce an aspatial action schedule over a 100-year horizon (10 periods of 10 years), and finally export a libCBM Standard Input Table (SIT) to simulate annual carbon stocks and fluxes.

Data and study area. The `tsa24_clipped` dataset is a clipped subset of a Timber Supply Area inventory used for demonstration. The initial inventory enumerates development types (dtypes)

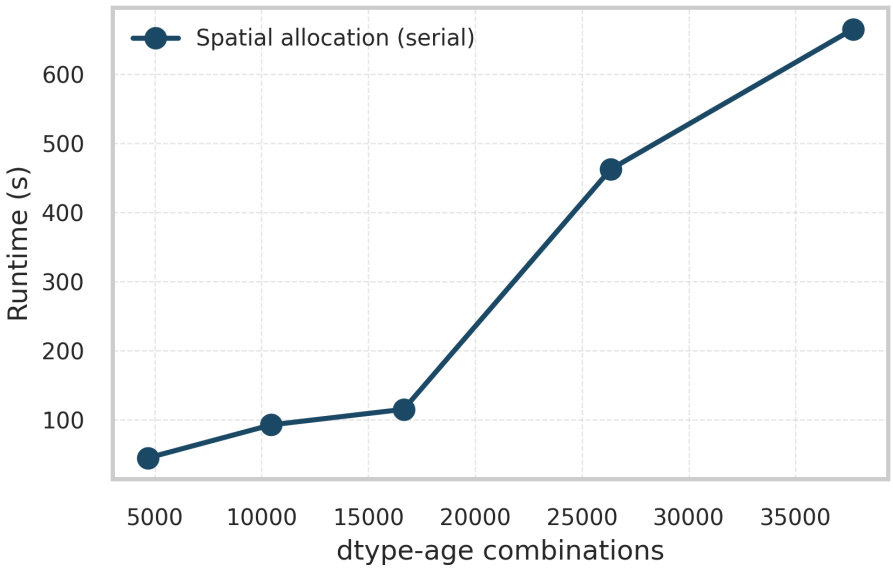


Figure 4: Spatial allocation runtime (serial `ForestRaster`) versus problem size after applying the heuristic schedule.

by theme tuples (e.g., analysis unit, species, site), with associated yield curves for total volume (`totvol`) and species-volume splits (`swdvol`, `hwdvol`). Actions and transitions define operability and state changes (e.g., harvest resets age).

Model setup and scheduling. We initialize the model with `base_year`=2020, `horizon`=10, `period_length`=10 y, and `max_age`=1000. After importing sections and initializing areas, we add a null action and reset actions. The area-control scheduler selects target areas by mask (default: AU-wise THLB) and applies actions using a priority-queue selector (oldest-first). Target areas are computed from area-weighted mean CMAI ages; utilization is set to 0.85 in the volume expression used for compiled flows. The resulting schedule is compiled and applied to produce period-by-period flows and stocks.

libCBM coupling and metrics. We define a disturbance-type mapping for actions (harvest→Clearcut harvesting without salvage; fire→Wildfire) and assign `last_pass_disturbance` per dtype to ensure consistent DOM spin-up. Using `ForestModel.to_cbm_sit` with softwood and hardwood volume names (`swdvol`, `hwdvol`), `admin_boundary` = `"British Columbia"`, and `eco_boundary` = `"Montane Cordillera"`, we export SIT config and tables. We then run libCBM for `n_steps`=200 years, ag-

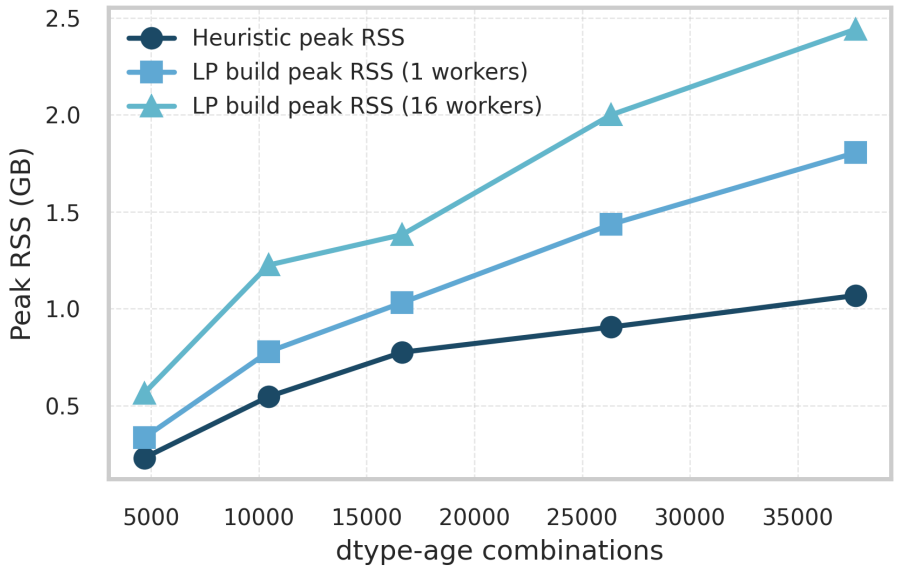


Figure 5: Peak resident memory for heuristic scheduling (serial) and LP model builds (1 and 16 workers).

gregating annual carbon stocks (biomass, DOM, total ecosystem).

Outputs. We report time-series of harvested area/volume and growing stock from WS3, and annual carbon stocks from libCBM. The reproduction package generates the figures and tables used below.

The full, deterministic workflow is scripted in `papers/ems/repro/generate_case_study.py`, which produces the flows table and carbon stocks table:

- `papers/ems/tables/scenario_flows.csv`: harvest area, harvest volume, and growing stock by period.
- `papers/ems/tables/annual_carbon_stocks.csv`: annual biomass, DOM, and total ecosystem carbon.

Figures 6 and 7 visualize the resulting flows and carbon stocks. The entire pipeline, including the architecture diagram (Figure 1), workflow pseudocode (Listing 1), and the spatial allocation example (Figure 2), is reproducible via `papers/ems/repro/make_repro.sh`. The libCBM run length (200 years) matches the example and can be adjusted.

Table 1: Peak resident memory during heuristic scheduling and LP model builds on the reference container. LP measurements report the maximum RSS observed while compiling the Model I matrix with 1 or 16 workers; spatial allocation runs serially.

Combo	$n_{\text{dtype-age}}$	Heuristic peak RSS (GB)	LP peak RSS (GB; 1 worker)	LP peak RSS (GB; 16 workers)
`tsa08`	4 682	0.23	0.33	0.56
`tsa08+tsa40`	10 453	0.52	0.78	1.22
`tsa08+tsa40+tsa41`	16 653	0.77	1.03	1.33
`tsa08+tsa40+tsa41+tsa24`	26 346	0.87	1.38	1.87
`tsa08+tsa40+tsa41+tsa24+tsa16`	37 691	1.07	1.81	2.51

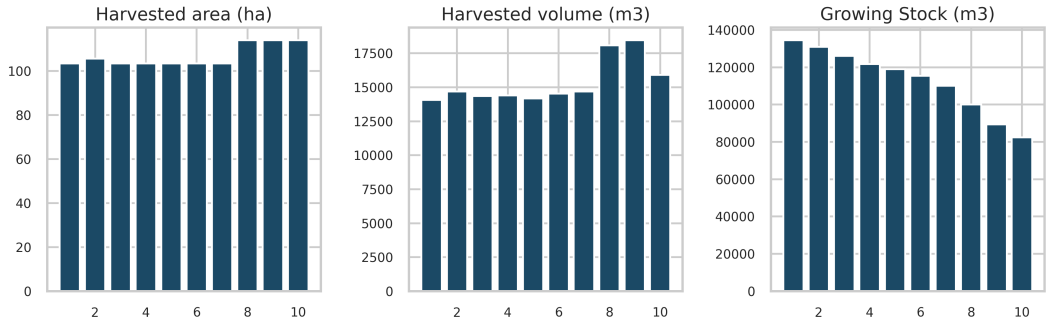


Figure 6: WS3 results: harvested area, harvested volume (utilization-adjusted), and growing stock over planning periods.

Results. The heuristic scheduler delivers a 10-period harvest plan whose flows stay within ˜10% of their period means without any explicit even-flow constraint: harvest area averages 106.8 kha and ranges from 103 to 114 kha, while harvest volume averages 15.3 Mm^3 with a 14.1–18.5 Mm^3 span (Table 2; Figure 6). Growing stock declines smoothly from 135 to 82 in the same area-equivalent units (consistent with the 0.85 utilization factor), documenting the state trajectory exported to libCBM. In the carbon stage, libCBM aggregates show biomass pools increasing from 2.7×10^5 tC to 3.1×10^5 tC over 200 years, while DOM remains within 1.7–1.9 $\times 10^5$ tC, yielding a total ecosystem carbon gain of 42 ktC (Figure 7). The summary tables `scenario_flows.csv` and `annual_carbon_stocks.csv` (`papers/ems/tables`) retain the raw outputs for external verification.

Table 2: Sequential demonstration parameters and configuration (WS3 → libCBM pipeline).

Setting	Value
Dataset	`tsa24_clipped` (Woodstock-format inputs; `examples/data/woodstock_model_files`)
Base year	2020
Planning horizon	10 periods (100 years)
Period length	10 years
Max age class	1000 years
Yield names	`totvol` (total volume), `swdvol` (softwood), `hwdvol` (hardwood)
Scheduling method	Area-control heuristic (priority-queue, oldest-first); utilization 0.85
Disturbance mapping (CBM)	harvest→Clearcut harvesting without salvage; fire→Wildfire
CBM export	`ForestModel.to_cbm_sit` (admin: British Columbia; eco: Montane Cordillera)
CBM run length	200 annual steps
Outputs	`scenario_flows.csv` (periodic flows/stocks); `annual_carbon_stocks.csv` (annual pools)

Reproducibility. This demonstration is intended to be transparent and portable; all input data reside in `examples/data/woodstock_model_files`. The spatial allocation step renders the raster-based harvest allocation shown in Figure 2, derived directly from the aspatial schedule via the ForestRaster utility. The folder papers/ems/repro contains `make_repro.sh`, which provisions a clean virtual environment, installs pinned dependencies (including the optional libCBM extra via `pip install ws3[cbm]`), and runs deterministic scripts that regenerate Figures 1–7 and the accompanying CSV tables. Outputs are checksum-stable across repeated runs, providing an audit trail for review and reuse.

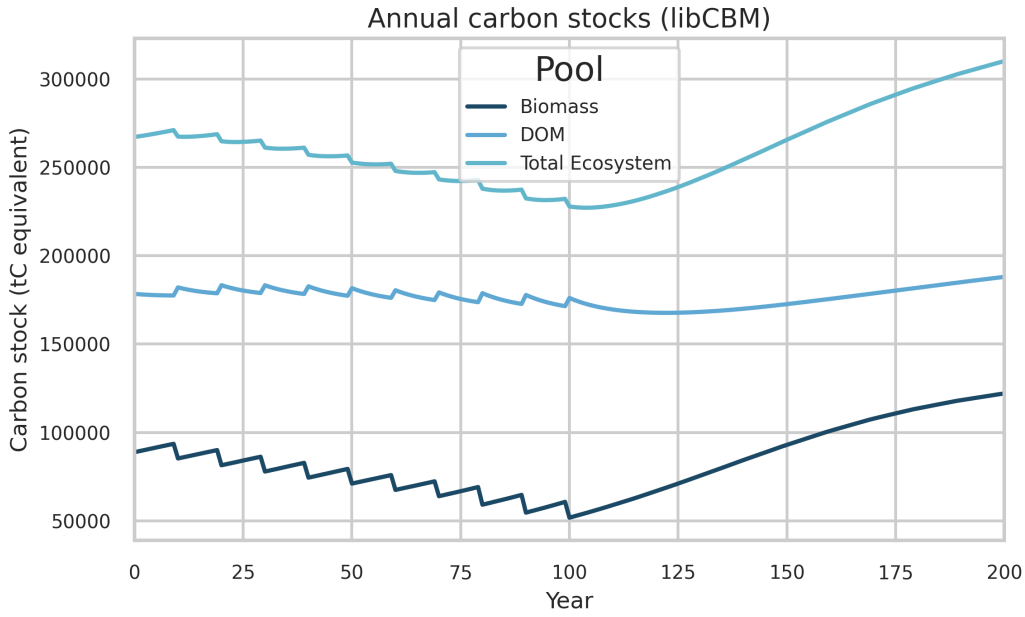


Figure 7: libCBM results: annual carbon stocks (biomass, dead organic matter, total ecosystem) over 200 years.

Broader example library and carbon-aware optimization

Beyond the sequential carbon-accounting demonstration, WS3 includes a curated set of executable notebooks designed as progressive tutorials, from synthetic examples to realistic policy-oriented analyses. Example 040 showcases carbon-aware optimization by embedding carbon prices and/or constraints directly in the harvest scheduling LP and computing carbon stocks and fluxes with `libcbm_py`. A public, end-to-end use-case demonstration exploring carbon-aware scheduling in a stylized British Columbia landscape is available in the repository by Yan (MSc), including scenario automation, sensitivity analysis, and reproducible outputs [40, 39].

4. Impact and reusability

WS3 enables transparent, auditable analysis for forest landscape planning and climate mitigation, responding to calls for credible natural climate solutions and land-sector mitigation accounting [13, 12]. It has supported research on bioenergy [7], value-creation potential and supply modelling [31, 30], and climate impact assessment [34, 20, 40]. WS3 integrates with the SpaDES ecosystem

for spatial simulation [8] and has been deployed as a backend in decision-support applications. The open MIT license, modular API, and PyPI distribution lower barriers to community contributions and reuse across jurisdictions.

Major use-cases (2015–2025).

- Strategic wood supply and bioenergy planning in mixed-wood forests: optimization and scenario analysis of value-creation options [7].
- Hybrid simulation–optimization for value indicators and model retrofitting: methods and transfer to production-style analyses [31, 30].
- Climate mitigation and carbon-aware planning in British Columbia: sequential WS3→libCBM pipelines and optimization under policy constraints; graduate theses and applied demonstrations [20, 40, 39].
- Decision-support prototypes for nature-based solutions: avoided fire and avoided harvest workflows (Examples 050/060) with net-emission indicators derived from libCBM.
- Integration with spatial simulation: `spades_ws3` module bridging WS3 with SpaDES for disturbance and landscape dynamics studies [8, 37].
- Application backends: WS3 used in web-based decision-support systems (e.g., `ecotrust_dss`) for scenario configuration and reporting [36].
- Ongoing initiatives: CCCANDiES (integrated carbon and ecosystem services; in progress) and mining-sector nature-based solutions (separate paper under review, Journal of Cleaner Production).

Interoperability fosters reusability: tabular/text inputs, raster outputs, and the libCBM linkage allow WS3 to slot into existing analytical pipelines. The reproduction package, unit tests, and CI enhance trust and facilitate method transfer, aligned with community guidance on open and efficient scientific practice [17, 24]. The architecture is intentionally general, making it applicable to studies of sustainable yield, carbon accounting, biodiversity, and policy trade-offs [2].

Practical impact and reusability highlights.

- Openness and packaging: MIT license, PyPI distribution, versioned releases archived on Zenodo [29]; complete examples and tutorials.
- Interoperability: Woodstock-format import for legacy/industry pipelines; libCBM SIT export for carbon; standard geospatial formats (GeoTIFF, shapefile).
- Reproducibility: deterministic reproduction package (`papers/ems/repro`) that regenerates all figures/tables; CI runs tests and validates examples.
- Extensibility: modular API for adding outputs/constraints and new scheduling logic (heuristic or LP-based); solver-agnostic (HiGHS by default; Gurobi optional).
- Integration pathways: SpaDES ecosystem via `spades_ws3`; backend for decision-support applications and web services.
- Education and onboarding: progressive example notebooks (from `010` to `060`) facilitate training and method transfer.

An advanced example notebook demonstrates how WS3 can be used to construct a stand-level optimization problem with explicit integration of carbon dynamics using `libcbm_py`. This example solves a linear program that balances timber harvest revenues with carbon sequestration benefits under different carbon pricing assumptions (see Example 040). The workflow illustrates how WS3's optimization layer and libCBM coupling enable carbon-aware planning experiments without bespoke glue code, supporting emerging policy analyses in nature-based climate solutions.

5. Comparison to alternatives

WS3 differs primarily in its open Python API, explicit libCBM integration, and hybrid spatial/aspatial workflow designed for reproducible research and policy analysis. We emphasize that SIMFOR operates at stand level, while WS3, Woodstock, Patchworks, Heureka, and JLP target landscape-scale planning with varying degrees of spatial explicitness and availability [32, 35, 8, 18, 27, 33].

Table 3: Comparison of WS3 with selected tools.

Tool	Scope	License	Optimization integration	Spatial workflow	Carbon integration	Geography focus
WS3	Landscape	Open (MIT)	Built-in (PuLP; optional Gurobi)	Hybrid aspatial-to-raster	Built-in libCBM linkage	General
Remsoft Woodstock	Landscape	Proprietary	Built-in	Extensions; connectors	External integrations	General
Patchworks	Landscape	Proprietary	Built-in	Spatial planning emphasis	External integrations	General
SpaDES	Simulation framework	Open	External packages	Fully spatial (agent-based, raster)	Via modules	General
SIMFOR	Stand-level	Open	Limited/none	N/A	Limited/none	General
Heureka	Landscape	Mixed/Restricted	Built-in	Spatial capabilities	External integrations	EU-focused
JLP (MELA)	Landscape	Restricted/Legacy	Built-in	Spatial extensions	External integrations	Finland-focused

Comparison narrative. Proprietary landscape systems such as Woodstock and Patchworks provide mature optimization and spatial planning capabilities, but their licensing and closed codebases limit transparent methods development, automated testing, and reproducibility at scale. SpaDES, by contrast, is an open simulation framework oriented to spatial processes and agent-based dynamics; it does not natively provide a solver-backed harvest scheduling layer but interoperates with WS3 via `spades_ws3` for hybrid workflows. Heureka and JLP are landscape-oriented and include optimization, yet are region-specific or legacy-maintained, which constrains adoption beyond their primary jurisdictions.

WS3's distinctive contribution is to combine: (i) a solver-agnostic, path-based Model I LP generator embedded directly in an open forest estate API; (ii) a documented, first-class linkage to libCBM for carbon stocks and fluxes; and (iii) a hybrid spatial/aspatial pipeline with standard geospatial I/O. These design choices enable reproducible, auditable workflows in notebooks and scripts, fast iteration on objectives/constraints, and easy integration into larger research and decision-support ecosystems. For many research and policy analyses, this balance of openness, extensibility, and end-to-end reproducibility is decisive—even when spatial planning itself is conducted in a specialized tool, WS3 can serve as the transparent scheduling and carbon accounting engine feeding it.

6. Limitations and scope

WS3 focuses on deterministic, aspatial forest estate formulations. The Model I LP implementation is well suited to even-flow policies, policy bounds, and value-maximisation problems, yet it inherits the classic limitations of the formulation: stochastic or endogenous disturbances cannot be represented faithfully without departing from linearity, and non-linear objectives or constraints require either linearisation or external wrappers. In practice, we rely on the heuristic scheduler to interleave random disturbances or sensitivity analyses when scenario logic demands it, and reserve the LP for static policy experiments.

Spatial representation is delivered through a post-hoc raster allocation step that respects operability and transitions but does not enforce spatial adjacency, road-building, or block-shape constraints. Those dynamics fall outside the scope of WS3 and are the motivation for ongoing spatial extensions (e.g., the forthcoming WS4 prototype). Users requiring spatial optimisation should therefore treat WS3 as a scheduling and accounting engine that feeds specialised spatial tools.

Finally, large LP instances involve substantial memory pressure—tens of gigabytes for the largest benchmarks—because the column generation compiles full path matrices. The reproduction package flags the LP scaling experiment as opt-in so that teams can execute it on appropriately provisioned infrastructure. All supporting assets, however, remain deterministic and auditable even when the LP portion is skipped.

7. Conclusions and future work

WS3 provides an open, extensible framework for integrating harvest scheduling, simulation, and carbon accounting. By coupling solver-backed scheduling with libCBM via a documented interface, WS3 supports policy-relevant analyses with transparent, reproducible workflows.

Future work will expand uncertainty handling (stochastic scheduling, sensitivity analysis), add cloud/HPC deployment patterns, and strengthen spatial allocation and visualization. We also plan to enhance carbon–economics linkages and support machine learning surrogates for speedups in large scenario ensembles. The open design invites community extensions and cross-ecosystem applications.

Code and data availability

WS3 source code is publicly available at `https://github.com/UBC-FRESH/ws3` and distributed via PyPI (`pip install ws3`). Versioned releases are archived on Zenodo (DOI 10.5281/zenodo.17219651) [29]. Documentation and tutorials are available at `https://ws3.readthedocs.io`. A complete reproduction package for the sequential demonstration, including environment files, scripts, and figure-generation notebooks, is provided as supplementary material and in the release archive.

Reproducibility. All figures and tables are generated by scripts in `papers/ems/repro`. The `make_repro.sh` script creates an isolated environment, installs pinned dependencies (`requirements.txt`), and runs `generate_diagrams.py`, the spatial allocation workflow (`generate_spatial_allocation.py`), and the sequential demonstration script (`generate_case_study.py`). Outputs are written to `papers/ems/figs` and `papers/ems/tables`, with intermediate rasters cached under `papers/ems/repro/_spatial_cache`. The exact versions are preserved via the Zenodo-archived release.

FAIR compliance. Findable: versioned releases archived with DOI on Zenodo [29] and indexed on GitHub/PyPI. Accessible: open-source license (MIT) and public repository without authentication. Interoperable: open formats for tabular and raster data (CSV, GeoTIFF) and a documented API linking to libCBM. Reusable: pinned environments, CI-tested examples, detailed documentation, and a complete reproduction package accompanying this article.

CRediT authorship contribution statement

Gregory Paradis: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing—original draft, Writing—review & editing, Visualization, Supervision, Project administration, Funding acquisition.

Acknowledgements

Development of WS3 was supported by the Forest Resources Environmental Services Hub (FRESH) at the University of British Columbia. Funding was provided by the Canada Foundation for Innovation (CFI JELF), the British Columbia Knowledge Development Fund (BCKDF), Mitacs, Environment and Climate Change Canada (ECCC), the Natural Sciences and Engineering

Research Council of Canada (NSERC), Natural Resources Canada (NRCan), and the Government of British Columbia. The author thanks Elaheh Ghasemi, Salar Ghotb, and Kathleen Coupland for their contributions to documentation, testing, and code quality improvements leading up to this release.

Declaration of competing interest

The authors declare no competing interests.

References

[1] Apex Resource Management Solutions, 2024. Syncrosim: Scenario modeling platform. Https://syncrosim.com (accessed 2025-09-28).

[2] Boisvenue, C., Paradis, G., Eddy, I.M., McIntire, E.J., Chubaty, A.M., 2022. Managing forest carbon and landscape capacities. Environmental Research Letters 17, 114013.

[3] Canadell, J.G., Monteiro, P.M.S., Joos, F., et al., 2022. Agriculture, forestry and other land use, in: Shukla, P.R., Skea, J., Slade, R., et al. (Eds.), Climate Change 2022: Mitigation of Climate Change. Contribution of Working Group III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change. Cambridge University Press, Cambridge, UK, pp. 751–850.

[4] Canadian Forest Service, 2025a. libcbm-py: Python implementation of the carbon budget model. Python package. https://pypi.org/project/libcbm/ (accessed 2025-09-28).

[5] Canadian Forest Service, 2025b. libcbm_py: Carbon budget model python interface. `https://github.com/cat-cfs/libcbm_py`. Source code repository. https://github.com/cat-cfs/libcbm_py (accessed 2025-09-28).

[6] Canadian Forest Service, 2025c. libcbm_py documentation. `https://cat-cfs.github.io/libcbm_py/`. Documentation site. https://cat-cfs.github.io/libcbm_py/ (accessed 2025-09-28).

[7] Cantegril, P., Paradis, G., LeBel, L., Raulier, F., 2019. Bioenergy production to improve value-creation potential of strategic forest management plans in mixed-wood forests of eastern canada. Applied Energy 247, 171–181.

[8] Chubaty, A.M., McIntire, E.J.B., et al., 2024. Spades: Spatially explicit discrete event simulation. R package and framework. https://spades.predictiveecology.org (accessed 2025-09-28).

[9] Daigneault, A.J., Hayes, D.J., Fernandez, I.J., Weiskittel, A.R., 2022. Forest carbon accounting and modeling framework alternatives: an inventory, assessment, and application guide for eastern us state policy agencies. Final Report , 32.

[10] Food and Agriculture Organization of the United Nations, 2020. Global Forest Resources Assessment 2020. Main Report. Technical Report. Rome. doi:10.4060/ca9825en.

[11] Forest Biometrics Research Institute, 2024. Forest projection system (fps) atlas. Https://www.forestbiometrics.com/fps/ (accessed 2025-09-28).

[12] Grassi, G., Conchedda, G., Federici, S., et al., 2022. Historical carbon fluxes from land use and land cover change and their uncertainty. Nature Climate Change 12, 734–742.

[13] Griscom, B.W., Adams, J., Ellis, P.W., et al., 2017. Natural climate solutions. Proceedings of the National Academy of Sciences 114, 11645–11650.

[14] Gunn, E., 2007. Models for strategic forest management, in: Weintraub, A., Romero, C., Bjørndal, T., Epstein, R. (Eds.), Handbook of Operations Research in Natural Resources. Springer Science+Business Media. chapter 16, pp. 317–341.

[15] Gurobi Optimization, LLC, 2024. Gurobi Optimizer Reference Manual. Https://www.gurobi.com.

[16] Hall, J., et al., 2023. HiGHS: High performance software for linear optimization. Software. https://highs.dev/ (accessed 2025-09-28).

[17] Hampton, S.E., Anderson, S.S., Bagby, S.C., et al., 2015. The tao of open science for ecology. Ecosphere 6, 1–13.

[18] Heureka Consortium, 2024. Heureka forestry decision support system. Https://www.heurekaslu.se/ (accessed 2025-09-28).

[19] Johnson, K.N., Scheurman, H.L., 1977. Techniques for prescribing optimal timber harvest and investment under different objectives: discussion and synthesis. Forest Science 23, a0001–z0001.

[20] Ke, J., 2024. Climate impact assessment under different forest harvesting and fertilization scenarios. Master's thesis. University of British Columbia.

[21] Kurz, W., Dymond, C., Stinson, G., Rampley, G., et al., 2009. Cbm-cfs3: A model of carbon-dynamics in forestry and land use change. Canadian Journal of Forest Research 39, 495–509.

[22] LANDIS-II Foundation, 2024. Landis-ii forest landscape model. Https://www.landis-ii.org (accessed 2025-09-28).

[23] Law, B.E., Hudiburg, T.W., Berner, L.T., 2018. Land use strategies to mitigate climate change in carbon dense temperate forests. Proceedings of the National Academy of Sciences 115, 3663–3668.

[24] Lowndes, J.S.S., Best, B.D., Scarborough, C., et al., 2017. Our path to better science in less time using open data science tools. Nature Ecology & Evolution 1, 0160.

[25] McIntire, E.J., Chubaty, A.M., Cumming, S.G., Andison, D., Barros, C., Boisvenue, C., Haché, S., Luo, Y., Micheletti, T., Stewart, F.E., 2022. Perfict: A re-imagined foundation for predictive ecology. Ecology Letters 25, 1345–1351.

[26] Mitchell, S., et al., 2023. PuLP: A Linear Programming Toolkit for Python. Software. https://coin-or.github.io/pulp/ (accessed 2025-09-28).

[27] Natural Resources Institute Finland (Luke), 2024. Jlp (mela) forest planning system. Http://mela2.metla.fi/mela/j/jlp-en.htm (accessed 2025-09-28).

[28] Nguyen, D., Henderson, E., Wei, Y., 2022. Prism: A decision support system for forest planning. Environmental Modelling & Software 157, 105515.

[29] Paradis, G., 2025. ws3: a wood supply simulation system. URL: `https://doi.org/10.5281/zenodo.17219651`, doi:`10.5281/zenodo.17219651`.

[30] Paradis, G., Lebel, L., 2018a. Estimating the value-creation potential of wood supply using a hybrid simulation-optimization approach. Technical Report CIRRELT-2018-24. Centre interuniversitaire de recherche sur les réseaux d'entreprise, la logistique, et le transport (CIRRELT). URL: `https://www.cirrelt.ca/documentstravail/cirrelt-2018-24.pdf`.

[31] Paradis, G., Lebel, L., 2018b. Retro-Fitting Value-Creation Potential Indicators to Long-Term Supply Models. CIRRELT-2018-23. URL: https://www.cirrelt.ca/documentstravail/cirrelt-2018-23.pdf.

[32] Remsoft Inc., 2025. Woodstock. Software. https://www.remsoft.com/products/woodstock/ (accessed 2025-09-28).

[33] SIMFOR Project, 2020. Simfor: A stand-level simulation tool. Project site. (accessed 2025-09-28).

[34] Smyth, C., Xu, Z., Lemprière, T., Kurz, W., 2020. Climate change mitigation in British Columbia's forest sector: GHG reductions, costs, and environmental impacts. Carbon Balance and Management 15, 1–22.

[35] Spatial Planning Systems Inc., 2025. Patchworks Forest Planning System. Software. https://www.spatial.ca/patchworks/ (accessed 2025-09-28).

[36] UBC-FRESH Lab, 2024a. ecotrust_dss: Ws3-backed decision support application. https://github.com/UBC-FRESH/ecotrust-dss. GitHub repository.

[37] UBC-FRESH Lab, 2024b. spades_ws3: Linking ws3 harvest scheduling with spades. https://github.com/UBC-FRESH/spades_ws3. GitHub repository.

[38] UBC FRESH Lab, 2024. Ws3 continuous integration workflow. https://github.com/UBC-FRESH/ws3/blob/main/.github/workflows/ci.yml. Accessed 2025-09-28.

[39] Yan, Y., 2024. msc_walter_yan: Carbon-aware harvest scheduling case study. GitHub repository. https://github.com/UBC-FRESH/msc_walter_yan (accessed 2025-09-28).

[40] Yan, Y., 2025. A Mathematical Modeling Framework to Optimize the Climate Change Mitigation Potential of the Forest Sector in British Columbia, Canada. Master's thesis. University of British Columbia.

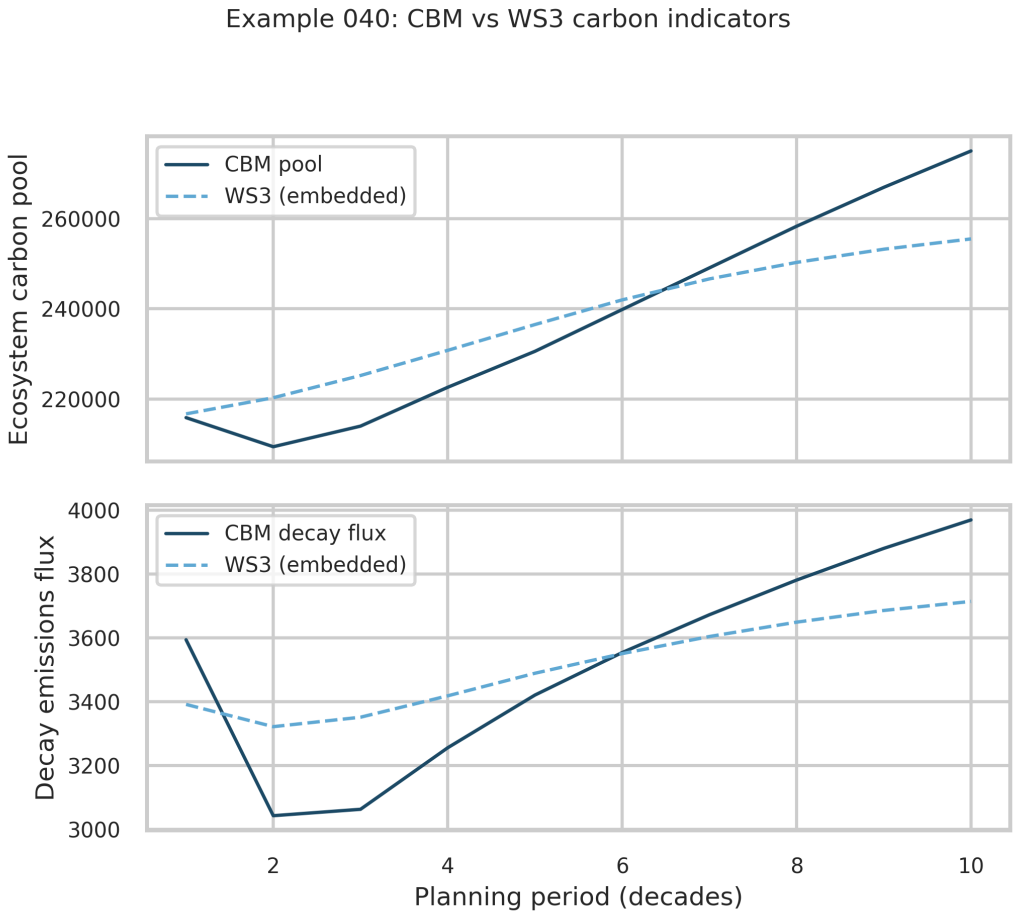


Figure 8: Neilson-hack demonstration (Example 040): comparison of libCBM (solid) and WS3-embedded (dashed) carbon indicators after bootstrapping pools/fluxes into WS3 as yield curves. Top: ecosystem carbon pools; Bottom: aggregate decay emissions flux.